



Python 객체지향



1. 함수
2. 함수 고오급편
3. 객체지향 프로그래밍
4. 객체지향 프로그래밍 고오급편

▼ 1. 함수

함수는 특정 작업을 수행하는 코드 묶음입니다.
자주 사용하는 코드를 함수로 만들어두면, 필요할 때마다 편리하게 재사용할 수 있습니다 📁

▼ 함수란?

```
def 함수 이름(매개변수1):
```

```
    실행할 코드
```

```
    return 결과값
```

```
def 레시피 이름(재료):
```

```
    조리방법
```

```
    return 음식 결과
```



함수란, 특정한 작업을 수행하는 코드 묶음입니다.

- **함수 이름:** 조리나 레시피의 이름처럼, 어떤 작업을 하는지 알려주는 이름입니다 
- **매개 변수:** 요리의 재료처럼, 함수가 작업하는데 필요한 재료입니다 
- **return:** 완성된 요리처럼, 함수의 실행 결과물입니다 

```
def greet(name):
```

```
    return f"안녕하세요, {name}님!"
```

```
message = greet("김기범")    # "안녕하세요, 김기범님!"
```

▼ 함수 이름

변수 이름과 마찬가지로, 함수의 이름은 함수가 수행하는 작업을 명확하게 설명해야 합니다.

- **숫자, 문자, 언더스코어(_)**의 조합만으로 함수 이름을 작성해야 하며, **숫자로 시작할 수 없습니다.**
- **동사로 시작하기:** calculate_sum(), print_message(), get_user_info() 등
- **소문자 사용:** 파이썬에서는 함수 이름을 소문자로 작성하는 것이 관례입니다
- **언더스코어(_) 사용:** 여러 단어를 연결할 때는 언더스코어를 사용합니다

```
def get_user_name():    # 좋은 예시
```

```
    return "Kim"
```

```
def UserName():          # 좋지 않은 예시
    return "Kim"
```



함수 이름만 보고도 그 기능을 쉽게 이해할 수 있도록, 명확하고 구체적으로 작성하는 것이 중요합니다.

▼ 프로그래밍이라고 하는 것

- 데이터 (자료)
 - 변수: 데이터에 이름을 붙여서 담을 수 있는 그릇
 - ex) 숫자, 문자, 불리언, list, dict
- 동작 (문제풀기)
 - 함수: 코드 동작들에 이름을 붙여서 담을 수 있는 그릇
 - ex) for, while, 소쿠리전략, 챔피언전략

▼ 매개 변수 (Parameters)

```
def 함수이름(매개변수1, 매개변수2, 매개변수3):
    실행할 코드
    return 결과값
```



매개변수는 함수에 전달되는 입력값으로, 함수 내에서 변수로 활용될 수 있습니다.

▼ 1. 기본적인 매개변수 만들기

```
def add(x, y):
    return x + y

result = add(3, 5)    # x와 y 값을 꼭 넣어줘야 합니다.
```

- 매개변수는 함수가 작업을 수행하는데 필요한 입력값입니다.
- 코드를 재사용하고 유연하게 만드는데 중요한 역할을 합니다.

▼ 2. 매개변수 전달 방식

- **위치 매개변수 (Positional Parameters):** 함수 정의된 순서대로 값을 전달하는 방식입니다.

```
def greet(name, age):  
    return f"{name}님은 {age}세입니다."  
  
# 순서대로 전달  
result = greet("김철수", 25)
```

- **이름 매개변수 (Keyword Parameters):** 매개변수의 이름을 지정해서 값을 전달하는 방식입니다.

```
# 이름으로 지정하여 전달  
result = greet(age=25, name="김철수")    # 순서가 바뀌어도 됨
```

▼ 3. 기본값 매개변수

- **기본값 설정:** 매개변수에 기본값을 지정하면, 값을 전달하지 않았을 때 기본값이 사용됩니다.

```
def order_coffee(type="아메리카노", size="중간"):  
    return f"{size}사이즈 {type} 주문완료"  
  
print(order_coffee())                # "중간사이  
즈 아메리카노 주문완료"  
print(order_coffee("라떼"))          # "중간사이  
즈 라떼 주문완료"  
print(order_coffee("카푸치노", "큰")) # "큰사이  
즈 카푸치노 주문완료"
```

▼ 결과값 (Return)

함수의 결과값(return)은 함수가 실행을 완료한 후 반환하는 값입니다.

▼ 1. 기본적인 결과값 특징

- **return 문을 만나면** 결과를 반환하면서 실행이 즉시 종료됩니다.
- **결과값이 없는 함수**는 자동으로 `None` 을 반환합니다.

```
def say_hello():
    print("안녕하세요!")
    # return 문이 없음

result = say_hello()    # None이 반환됨
```

▼ 2. 다중 결과값 반환

- **리스트, 딕셔너리 등의 자료구조**를 반환할 수 있습니다

```
# 딕셔너리 반환
def create_person(name, age):
    return {
        "name": name,
        "age": age,
        "adult": age >= 20
    }
```

- **여러 개의 값을 동시에 반환**할 수 있습니다 (튜플 형태로 반환)

```
def get_user_info():
    name = "김철수"
    age = 25
    city = "서울"
    return name, age, city    # 튜플로 반환

# 언패킹으로 받기
name, age, city = get_user_info()

# 튜플로 받기
info = get_user_info()
```

▼ 3. 조건부 결과값

- **조건에 따라 다른 값을 반환**할 수 있습니다.

```
def check_score(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    else:
        return "C"

grade = check_score(85)    # "B" 반환
```

▼ 4. 결과값의 활용

- 함수의 결과를 변수에 저장하여 재사용할 수 있습니다.
- 다른 함수의 입력값으로 사용할 수 있습니다.

```
def calculate_area(radius):
    return 3.14 * radius ** 2

def format_area(area):
    return f"원의 넓이는 {area}입니다."

# 함수의 결과값을 다른 함수의 입력으로 사용
radius = 5
area = calculate_area(radius)
result = format_area(area)
print(result)    # "원의 넓이는 78.50입니다."
```

▼ 실습

▼ 미니 실습

1. 문자열 끝에 물음표를 붙여주는 함수를 만들어봅시다.

- **입력:** 문자열
- **출력:** 맨 뒤에 물음표가 붙은 문자열
- **실행 예시:**
`add_question ("잘못되었습니다")` 을 실행하면 `잘못되었습니다?` 가 나와야 합니다.
- **힌트:** 두 문자를 합치려면 `+` 기호를 사용합니다.

▼ 코드 예시

```
# 1. 문자열 끝에 물음표를 붙여주는 함수를 만들어봅시다.

def add_question(text):
    return text + "?"

result = add_question("잘못들었습니다")
print(result)
```

2. 온도를 변환해주는 함수를 만들어봅시다.

- 입력: 화씨
- 출력: 섭씨
- 실행 예시:
`convert_to_celsius(32)` 을 실행하면 `0.0` 이 나와야 합니다.
- 힌트: 화씨를 섭씨로 바꾸는 공식은 $(\text{화씨} - 32) \times 5/9$ 입니다.

▼ 코드 예시

```
# 2. 온도를 변환해주는 함수를 만들어봅시다.

def convert_to_celsius(fahrenheit):
    celsius = (fahrenheit - 32) * 5/9
    return celsius

result = convert_to_celsius(32)
print(result)
```

▼ 피보나치 수열 🍗

- 피보나치 수열 다들 기억하시나요?
 - 피보나치 수열은 각 숫자가 바로 앞의 두 숫자의 합과 같은 수열입니다:
`1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...`
- 피보나치 수열은 어디에 쓰면 좋을까요?
 - `피보나치 수열의 n번째 수 == n명이 치킨을 먹을 때 주문하면 딱 좋은 마리수`
 - 예를 들어, 피보나치 수열을 다음과 같이 정의할 수 있습니다:

- 1명: **1마리** (한 명은 한 마리 먹을 수 있음)
 - 2명: **1마리** (두 명이 한 마리 나눠 먹을 수 있음)
 - 3명: **2마리** (세 명은 두 마리가 적당함)
 - 4명: **3마리** (네 명도 두 마리로 충분)
 - 5명: **5마리** (다섯 명은 세 마리가 필요)
 - 그 이후로는 피보나치 수열과 같은 패턴으로 증가
- 피보나치 수열의 n번째 구현하려면 아래와 같은 방법을 쓰면 됩니다:
 1. **현재 숫자**와 **이전 숫자**를 더합니다.
 2. 1번에서 더해 결과를 **현재 숫자**에 저장합니다.
원래 있던 **현재 숫자**는 **이전 숫자**에 저장합니다.
 3. 1-2번 과정을 n번 반복합니다.

▼ 2. 함수 고오급편

▼ 지역변수 vs 전역변수

변수는 크게 지역변수(**Local Variable**)와 전역변수(**Global Variable**)로 나눌 수 있습니다.

▼ 1. 지역변수 (Local Variable)

- 함수 내부에서 선언된 변수로, 해당 함수 안에서만 사용할 수 있습니다.
- 함수가 종료되면 자동으로 소멸됩니다.

```
def calculate_sum():
    result = 10 + 20    # result는 지역변수
    print(result)

calculate_sum()        # 30
print(result)          # NameError: result는 함수 밖에서
                        # 접근 불가
```

▼ 2. 전역변수 (Global Variable)

- 함수 외부에서 선언된 변수로, 프로그램 전체에서 접근할 수 있습니다.
- 함수 내부에서 전역변수를 사용하려면 `global` 키워드를 사용해야 합니다.

```
total = 0    # 전역변수

def add_to_total(num):
    global total    # 전역변수 사용 선언
    total += num    # 전역변수 수정

add_to_total(10)
print(total)    # 10
```

▼ 3. 변수의 범위 (Scope)

- 지역 범위 (Local Scope): 함수 내부에서만 접근 가능
- 전역 범위 (Global Scope): 프로그램 어디서나 접근 가능

```
apt = "로제의 아파트"    # 전역변수

def old_korean():
    apt = "윤수일의 아파트"    # 지역변수
    print(f"옛날 한국인들은 apt노래를 얘기하면 {apt}가 떠
    오릅니다")

def new_korean():
    global apt
    print(f"젊은 한국인들은 apt노래를 얘기하면 {apt}가 떠
    오릅니다")

old_korean()
new_korean()
```

▼ 4. 주의사항

- 가능하면 전역변수 사용을 최소화하는 것이 좋습니다.
- 전역변수를 과도하게 사용하면, 대체 누가 전역변수를 수정했는지 찾아내기 어려울 수 있습니다.
즉, 유지 보수성과 안정성이 떨어질 수 있습니다.

- 함수가 필요한 값은 매개변수로 전달받고, 결과는 return으로 반환하는 것이 좋습니다.

```
# 좋지 않은 예
total = 0
def add_numbers(a, b):
    global total
    total = a + b

# 좋은 예
def add_numbers(a, b):
    total = a + b
    return total

result = add_numbers(10, 20)
```

▼ 매개변수 고급편



가변 매개변수를 사용하면 여러 파라미터를 유연하게 만들고 처리할 수 있습니다.

▼ 1. 가변 매개변수

▼ * 매개변수명

여러 개의 위치 인자를 **튜플**로 받습니다.

```
def calculate_sum(*numbers):
    return sum(numbers)

print(calculate_sum(1, 2))           # 6
print(calculate_sum(1, 2, 3, 4, 5)) # 15
```

▼ ** 매개변수명

여러 개의 키워드 인자를 **딕셔너리**로 받습니다.

```
def print_info(**info):
    for key, value in info.items():
```

```

        print(f"{key}: {value}")

print_info(name="김철수", age=25, city="서울")
# 출력:
# name: 김철수
# age: 25
# city: 서울

```

▼ 2. 매개변수 조합 사용

여러 유형의 매개변수를 함께 사용할 수 있습니다. 단, 순서를 지켜야 합니다:

```

def print_profile(name, *hobby, city="서울", **pets):
    print(f"이름: {name}")
    print(f"취미: {hobby}")
    print(f"도시: {city}")
    print(f"반려동물: {pets}")

print_profile(
    "김기범",                                # name
    "기타", "드라이빙", "운동",             # *hobby
    city="경기",                             # city
    dog="곰희", cat="냐옹이"                 # **pets
)

```

▼ 실습

▼ 카페 주문 관리 시스템 만들기

```
menu = {"아메리카노": 3000, "카페라떼": 4000}
```

카페 주문을 받고 계산하는 간단한 시스템을 만들어봅시다.

카페 메뉴는 위와 같이 전역변수로 선언되어 있습니다.

▼ 1. `add_menu_item` 함수 만들기

- `menu` 변수에 새로운 메뉴를 추가하는 함수를 만들어봅시다.
- 입력값은 **메뉴이름**과 **메뉴가격**을 받도록 만들어줍니다.

▼ 실행 예시

```

menu = {"아메리카노": 3000, "카페라떼": 4000}

add_menu_item("카푸치노", 4500)

print(menu)      # {"아메리카노": 3000, "카페라떼": 4000, "카푸치노": 4500}

```

▼ 코드 예시

```

menu = {
    "아메리카노": 3000,
    "카페라떼": 4000
}

def add_menu_item(name, price):
    global menu
    menu[name] = price

add_menu_item("카푸치노", 4500)
print(menu)

```

▼ 2. `place_order` 함수 만들기

- 여러 메뉴들을 입력 받아서, 주문한 음료와 총합을 보여주는 함수입니다.
- **가변 매개변수**를 활용하여 여러 메뉴들을 입력 받을 수 있도록 만들어줍니다.
- 주문한 메뉴가 메뉴판에 없는 경우, `"메뉴에 없는 항목입니다."`를 출력해줍니다.

▼ 실행 예시

```

place_order("아메리카노", "카페라떼", "에스프레소")

place_order("카푸치노", "아메리카노")

```

▼ 출력 예시

주문한 항목: ['아메리카노', '카페라떼']
총 금액: 7000원
에스프레소는 메뉴에 없는 항목입니다.

▼ 코드 예시

```
def place_order(*orders):
    global menu                # 전역변수인 menu
    사용
    total_price = 0            # 총 금액
    not_existing_menu = []     # 메뉴에 없는 주문
    목록

    for order in orders:
        if order in menu:
            # 메뉴에 주문한 상품이 있을 때
            total_price += menu[order]
        else:
            # 메뉴에 주문한 상품이 없을 때
            not_existing_menu.append(order)

    print(f"주문한 항목: {orders}")
    print(f"총 금액: {total_price}원")
    print(f"{not_existing_menu}는 메뉴에 없는 항목입니다.")

place_order("아메리카노", "카페라떼", "에스프레소")
```

▼ 3. 객체지향 프로그래밍



객체지향 프로그래밍(**Object-oriented Programming**)은 **현대 프로그래밍**의 근간이 되는 기법입니다.
 거의 모든 프로그램들은 **객체지향**이라는 방법으로 개발되고 있다고 봐도 과언이 아닙니다.

▼ 객체란?



'객체'란 특정한 사물이나 개념을 프로그래밍으로 표현한 것입니다. (ex. 자동차, 학생, 계좌, 동물)

▼ 객체의 구성 요소

객체는 크게 두 가지로 구성됩니다.

1. 데이터

- 객체가 가지고 있는 정보나 특성입니다.
- 우리가 지금까지 배운 '변수'라는 개념으로 쉽게 표현할 수 있습니다.
- ex) 자동차: **차번호**, **연식**, **제조사**, **남은 연료**

2. 기능

- 객체가 할 수 있는 행동입니다.
- 우리가 지금까지 배운 '함수'라는 개념으로 쉽게 표현할 수 있습니다.
- ex) 자동차: **시동걸기**, **달리기**, **멈추기**, **경적울리기**

즉, 객체는 **변수와 함수를 하나로 묶어서** 세상에 존재하는 개념들을 프로그래밍으로 표현한 것입니다.

▼ 객체로 표현할 수 있는 것들 (예시)

▼ 구체적인 사물이나 생물

▼ 스마트폰

- 데이터: 모델명, 용량, 배터리
- 기능: 전화걸기, 앱 실행하기

▼ 커피머신

- 데이터: **브랜드**, **용량**, **온도**, **원두**

- 기능: 커피내리기, 세척하기

▼ 학생

- 데이터: 이름, 학번, 학년
- 기능: 공부하기, 시험보기

▼ 식당

▼ 추상적 개념

▼ 계좌

▼ 게임캐릭터

▼ 스케줄 일정

▼ 데이터

- 날짜, 시간
- 무엇을 하려고 만나는지 (목적)
- 장소

▼ 기능

- 저장하기
- 삭제하기
- 업데이트 하기
- 초대하기

▼ 시스템과 서비스

▼ 쇼핑카트

▼ 채팅방

- 데이터: 채팅 내용, 참여자 목록, 생성된 날짜, 채팅방 ID,
- 기능: 채팅 내용을 가져온다, 참여자를 강퇴, 방폭

▼ 음악 플레이어

▼ 설정



이 외에도 수 없이 많은 상라만상(?)들을 객체로 표현할 수 있습니다.
점심시간에 길을 걸으면서 사물들을 보면,
객체로 어떻게 표현할 수 있을 지 생각해봅시다. (데이터와 기능에 초점)

▼ 객체 지향이란?

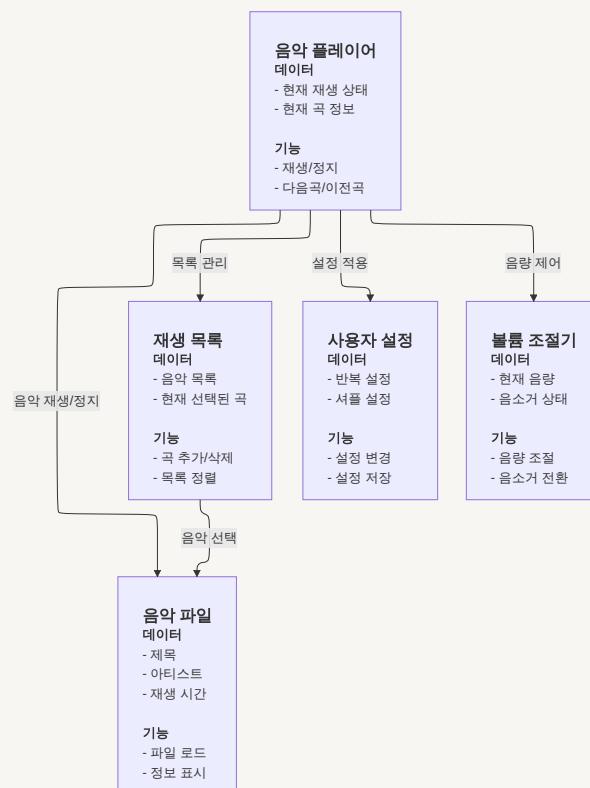


'객체지향'이란, 객체들끼리 서로 상호작용하며 프로그램을 구성하는 방식을 말합니다.

- 객체지향 프로그래밍의 핵심은 **현실 세계의 개념을 프로그래밍으로 표현**하는 것입니다.
- 이를 통해 직관적이고 유지보수가 쉬운 코드를 작성할 수 있습니다.

▼ 객체로 앱을 만드는 법 (음악 플레이어)

음악 플레이어 앱을 예시로 각 객체들이 어떻게 상호작용하는지 살펴봅시다:

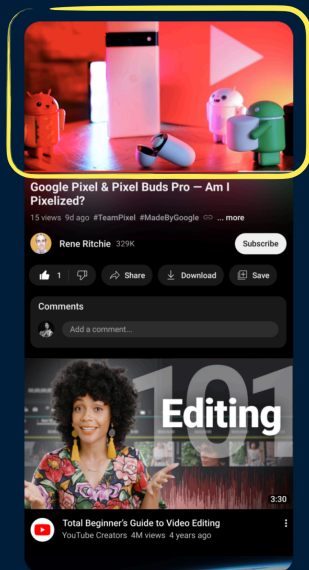
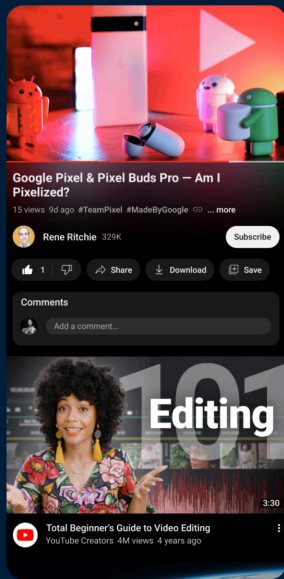


이렇게 각 객체들이 서로 메시지를 주고받으면서, 하나의 완성된 음악 플레이어 앱으로 동작하게 됩니다.

각 객체들의 구성을 살펴보면:

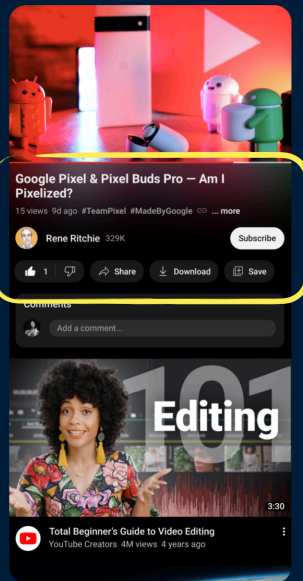
- **음악 플레이어 객체**
 - 데이터: 현재 재생 상태, 현재 곡 정보
 - 기능: 재생하기, 멈추기, 다음곡, 이전곡
- **재생 목록 객체**
 - 데이터: 음악 목록, 현재 선택된 곡
 - 기능: 곡 추가하기, 곡 삭제하기, 목록 정렬하기
- **음악 파일 객체**
 - 데이터: 제목, 아티스트, 재생 시간
 - 기능: 파일 불러오기, 정보 표시하기
- **사용자 설정 객체**
 - 데이터: 반복 설정, 셔플 설정
 - 기능: 설정 변경하기, 설정 저장하기
- **볼륨 조절기 객체**
 - 데이터: 현재 음량, 음소거 상태
 - 기능: 음량 조절하기, 음소거 전환하기

▼ 객체로 앱을 만드는 법 (유튜브)



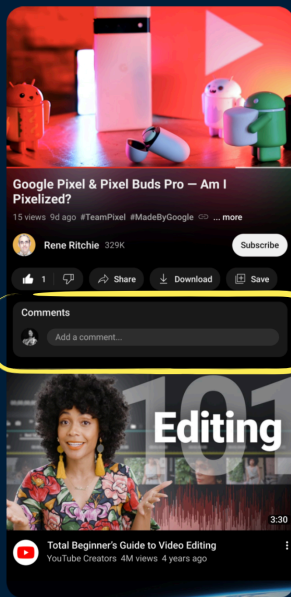
동영상 플레이어

- 데이터: 재생 여부, 동영상 파일, 재생 위치
- 기능: 일시정지, 다음 영상 보기



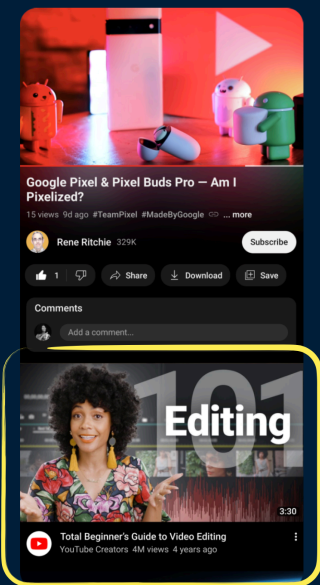
동영상 정보 섹션

- 데이터: 제목, 조회수
- 기능: 좋아요, 공유하기



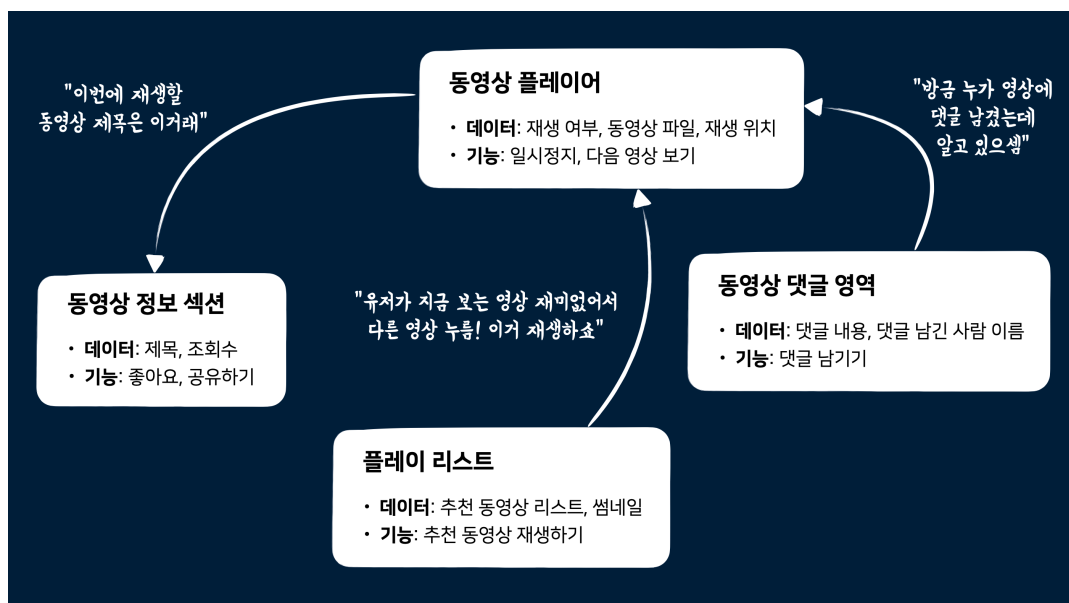
동영상 댓글 영역

- 데이터: 댓글 내용, 댓글 남긴 사람 이름
- 기능: 댓글 남기기



플레이 리스트

- 데이터: 추천 동영상 리스트, 썸네일
- 기능: 추천 동영상 재생하기



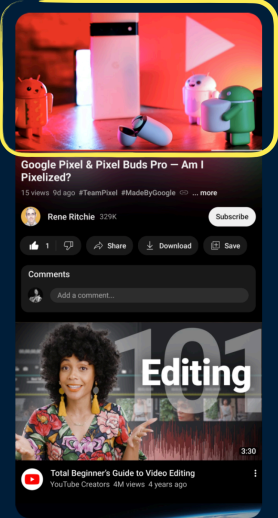
▼ 클래스: 객체를 만드는 틀



붕어빵 을 만들기 위해 붕어빵틀 이 필요한 것처럼,
건물 을 만들기 위해 설계도 가 필요한 것처럼,
객체 를 찍어내기 위해서는 객체를 만들기 위한 설계도 (Class) 가 필요하
입니다.

▼ 클래스란?

- 클래스는 객체를 대량생산 하기 위한 주형들입니다.
마치 붕어빵 틀로 붕어빵을 찍어내듯이, 클래스로 객체를 생성할 수 있습니
다.
- 클래스에는 객체가 가져야 할 데이터와 기능이 정의되어 있습니다.
- 클래스를 통해 만들어진 결과물을 '객체' 또는 '인스턴스 (instance)'라고 부릅
니다.
- 붕어빵마다 각각 다른 소스가 담길 수 있듯, 객체마다 다른 데이터가 담길 수
있다!



- 동영상 재생이 필요할 때마다, '붕어빵틀'에서 새로운 플레이어를 찍어냄!
- 물론, 매번 똑같은 플레이어가 만들어지는 건 아님
플레이어가 재생할 동영상 데이터는 바뀔 수 있음
- 마치 붕어빵을 만들 때마다 소스를 바꿀 수 있듯이!

▼ 클래스를 코드로 표현하기

▼ 특정한 기능을 가진 클래스 만들기

```
class Car:

    # 기능(메서드)
    def start(self):
        print("차가 출발합니다.")

    def stop(self):
        print("차가 멈춥니다.")

ferrari = Car()    # 객체, 인스턴스

ferrari.start()
ferrari.stop()
```

▼ 클래스 이름을 만드는 규칙

- 변수명: ferrari_car
- 클래스명: FerraiCar, CoffeeMachine

▼ 메소드

- 클래스가 가진 기능(Function, 함수, 동사)을 다른 말로 메소드 라고 부릅니다.

- 클래스로 실제로 객체를 만들어내면, 객체가 가진 **메소드를 실행**할 수 있습니다.

▼ 메서드 생성 규칙

- 메서드는 기본적으로 함수를 만드는 것과 동일합니다.
- 클래스 밑에 잘 넣어주고 들여쓰기를 해주면 됩니다.
- 대신 메서드는 첫 번째 매개변수로 항상 `self`를 받아야 한다는 특별한 규칙이 있습니다.

▼ 특정한 **데이터**를 가진 클래스 만들기

```
class Car:
    # 데이터 (인스턴스 변수)
    def __init__(self, brand, color):
        self.brand = brand          # brand 인스턴스 변수에 brand 저장
        self.color = color          # color 인스턴스 변수에 color 저장

    # 기능 (메서드)
    def start(self):
        print(f"{self.color}색 {self.brand}차가 출발합니다.")

    def stop(self):
        print(f"{self.color}색 {self.brand}차가 멈춥니다.")

ferrari = Car("페라리", "빨강")

print(ferrari.brand)
print(ferrari.color)

단팥붕어빵 = FishCake("단팥")
생크림붕어빵 = FishCake("생크림")
```

▼ 인스턴스 변수

- 클래스가 가진 **데이터**를 다른 말로 **인스턴스 변수** 라고 부릅니다.
- 인스턴스 변수는 말 그대로, **인스턴스(객체)** 가 가진 **고유한 데이터**입니다.
- 클래스로 실제로 객체를 만들어내면, 객체가 가진 인스턴스 변수를 참조할 수 있습니다.

▼ 생성자

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# 객체 생성
puppy = Dog("멍멍이") # Dog 클래스로부터 puppy 객체 생성
```

▼ **__init__** 메서드

- 객체가 생성될 때 자동으로 호출되는 특별한 메서드로서, **생성자** 라고도 부릅니다.
- 객체를 만들기 위해 **필요한 재료들을 파라미터로** 자유롭게 만들 수 있습니다.
- 첫번째 파라미터는 **self**를 적어줍니다.

▼ 생성자의 역할

1. 말 그대로 객체를 **생성** 해주는 역할을 합니다.
2. 클래스가 갖는 **데이터(인스턴스 변수)** 를 정의하고 초기화하는 역할을 합니다.

▼ 생성자의 기능

1. 개성 있는 객체를 만들 수 있다.
2. 객체한테 본인이 가진 개성을 물어볼 수 있다.
3. 객체로 하여금 조금 더 개성있는 동작을 하게 만들 수 있다.

▼ 생성자로 객체 찍어내는 법

- 생성자로부터 객체를 생성하는 방법은 매우 간단합니다.
클래스 이름을 적고 괄호를 붙여서 재료를 넣어주면 됩니다.

- 생성자에 매개변수가 정의되어 있다면, 괄호 안에 필요한 값들을 전달 해주어야 합니다.

▼ self란?

- 메서드의 첫번째 매개변수는 항상 `self` 를 사용합니다.
- `self` 는 메서드(동사)를 실행한 객체(주어)를 뜻합니다.
- 메서드를 호출할 때 `self` 는 자동으로 전달되므로, 직접 값을 넘겨줄 필요가 없습니다.
- `self` 를 통해 해당 주어의 다른 메서드나 변수에 접근할 수 있습니다.

```
class Dog:
    def __init__(self, name):
        self.name = name          # self.name은 객체
        의 인스턴스 변수

    def bark(self):               # self를 통해 해당
        객체가 가진 인스턴스 변수에 접근 가능
        print(f"{self.name}가 짖습니다: 멍멍!")

dog1 = Dog("시츄")
dog1.bark()                      # self는 자동으로
                                전달됨. dog1의 self는 시츄임.

dog2 = Dog("말티즈")
dog2.bark()                      # self는 자동으로
                                전달됨. dog2의 self는 시츄임.
```

▼ 실습

▼ 미니 실습

강아지 클래스 (`Dog`)를 만들어봅시다.

1. 강아지의 `이름`, `나이`, `종류`, `체력` 을 데이터로 갖도록 만들어봅시다.
 - 체력은 숫자로 표현합니다.
2. 강아지에게 `짖기(bark)` 기능을 추가해봅시다.
 - 짖기 기능을 실행하면 `체력이 20만큼` 닳습니다.

- 짖기 기능을 실행하면 **강아지의 이름을 활용하여** 아래와 같은 형태로 출력되어야 합니다.

"바둑이가 짖습니다. 뽕뽕!"

▼ 실행 예시

```
my_dog = Dog(name="바둑이", age=3, breed="말티즈", energy=100)
my_dog.bark()
```

bark 실행시 "바둑이가 짖습니다. 뽕뽕!" 이 출력되어야 함.

▼ 코드 샘플

```
class Dog:

    def __init__(self, name, age, breed, energy):
        self.name = name
        self.age = age
        self.breed = breed
        self.energy = energy

    def bark(self):
        self.energy = self.energy - 20
        print(f"{self.name}가 짖습니다. 뽕뽕!")

my_dog = Dog(name="바둑이", age=3, breed="말티즈", energy=100)

my_dog.bark()
```

3. 강아지에게 **잠자기 기능(sleep)** 을 추가해봅시다.

- 잠자기 기능을 실행하면 **체력이 +100만큼 회복**됩니다.
- 잠자기 기능을 실행하면 **강아지의 이름과 체력 정보를 활용하여** 아래와 같이 출력되어야 합니다.

"바둑이가 잠을 잡니다. Zzz... 체력이 200이 되었습니다."

▼ 실행 예시

```
my_dog = Dog(name="바둑이", age=3, breed="말티즈", energy=100)
my_dog.sleep()

# sleep 실행시 "바둑이가 잠을 잡니다. Zzz... 체력이 200이 되었습니다." 이 출력되어야 함.
```

▼ 코드 샘플

```
class Dog:

    def __init__(self, name, age, breed, energy):
        self.name = name
        self.age = age
        self.breed = breed
        self.energy = energy

    def bark(self):
        self.energy = self.energy - 20
        print(f"{self.name}가 짖습니다. 뽕뽕!")

    def sleep(self):
        self.energy += 100
        print(f"{self.name}가 잠을 잡니다. Zzz... 체력이 {self.energy}이 되었습니다.")

my_dog = Dog(name="바둑이", age=3, breed="말티즈", energy=100)

my_dog.bark()
my_dog.sleep()
```

4. 강아지에게 밥먹기 기능 (eat) 을 추가해봅시다.

- 밥먹기 기능을 실행하면 **체력이 +20만큼 회복**됩니다.
- 밥먹기 기능을 실행할 때는 **음식 이름** 을 넣을 수 있어야 합니다.
- 밥먹기 기능을 실행하면, **강아지 이름과 음식 이름을 활용하여** 아래와 같이 출력합니다.

"바둑이가 개껌을 먹습니다. 체력이 40이 되었습니다."

▼ 실행 예시

```
my_dog = Dog(name="바둑이", age=3, breed="말티즈", 체력=100)

my_dog.eat("개껌")

# eat 실행시 "바둑이가 개껌을 먹습니다. 체력이 140이 되었습니다."가 출력되어야 함
```

▼ 코드 샘플

```
class Dog:

    def __init__(self, name, age, breed, energy):
        self.name = name
        self.age = age
        self.breed = breed
        self.energy = energy

    def bark(self):
        self.energy = self.energy - 20
        print(f"{self.name}가 짖습니다. 웡웡!")

    def sleep(self):
        self.energy += 100
        print(f"{self.name}가 잠을 잡니다. Zzz... 체력이 {self.energy}이 되었습니다.")

    def eat(self, food):
        self.energy += 20
```



```
print(f"{self.name}가 {food}을 먹습니다. 체력이 {self.energy}이 되었습니다.")
```

```
my_dog = Dog(name="바둑이", age=3, breed="말티즈", energy=100)
```

```
my_dog.bark()  
my_dog.sleep()  
my_dog.eat("개껌")
```

▼ 4. 객체지향 프로그래밍 고오급편



객체는 단순히 데이터와 기능을 갖는 것을 넘어서, 현실 세계를 더 잘 표현하기 위한 몇 가지 기능을 더 가지고 있습니다.

▼ 상속(Inheritance)



상속이란, 마치 자식이 부모의 모습을 물려 받는 것처럼 기존 클래스의 기능을 물려 받고 확장하는 것을 의미합니다. 상속을 잘 사용하면 코드의 중복을 줄이고 체계적인 구조를 만들 수 있습니다.

▼ 상속이란?

- 마치 자식이 부모의 모습을 물려 받는 것처럼, 기존 클래스의 **데이터 (속성)** 과 **메소드 (행동)** 을 물려받아 새로운 클래스를 만들 수 있습니다.

▼ 상속의 예시

'동물' 클래스를 상속받아 '강아지'와 '말티즈' 클래스를 만들 수 있습니다.

- **동물**: 움직인다, 먹고, 소리를 낸다.
- **강아지**: 짖기, 산책하기 등의 특별한 기능이 추가됨

- 말티즈: 강아지의 모든 특성을 물려받고, 양칼지다는 고유한 특징이 추가됨

▼ 상속의 특징

1. 부모 클래스의 데이터 (속성) 과 메소드 (행동) 을 물려 받을 수 있습니다.

▼ 예시

모든 음악가들은 '사람'이라는 특성을 갖습니다.

즉, '이름'과 '나이' 정보를 갖고, '호흡'이라는 동작을 할 수 있습니다.



2. 자식 클래스는 부모로부터 물려 받은 데이터 (속성) 는 바꿀 수 없습니다.

▼ 예시

부모로부터 물려 받은 DNA나 성씨는 유지가 됩니다.

3. 자식 클래스는 부모로부터 물려 받은 메소드 (행동양식) 은 바꿀 수 있습니다.
(Override)

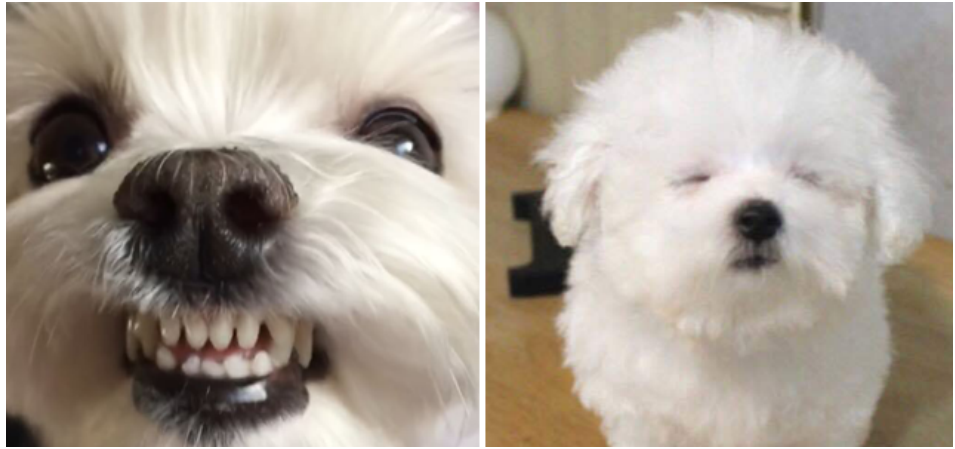
▼ 예시



강아지는 동물로서 '짖기'를 할 수 있지만,
'짖을 때' 멍멍🐶 소리를 냅니다.

4. 자식 클래스는 부모로부터 물려 받은 메소드 외에도, 완전히 새로운 데이터
(속성) 메소드 (행동양식) 을 가질 수 있습니다.

▼ 예시



말티즈는 강아지로서의 모든 특징과 행동을 물려 받았지만,

'참지 않는' 독특한 버릇이 추가됐습니다.

- ▼ 상속 구조를 잘 만드는 법?
- ▼ 이 중에 두 개를 묶는다면?



사물에 대한 기존의 선입견을 버리고,
객관적으로 보고 상속 구조를 만들 수 있어야 합니다.

- ▼ 이 동물은 무슨 과에 속할까요?



▼ 실제 코드



상속을 하려면 클래스 이름 옆에 부모 클래스 이름을 적습니다.

class 자식클래스(부모클래스):

```
# 부모 클래스 정의
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        print("동물이 소리를 냅니다")
```

```

# 자식 클래스 정의
class Dog(Animal):
    # Animal 클래스를 상속
    def make_sound(self):
        # 메소드 오버라이딩
        print(f"{self.name}가 멍멍!")
        super() #부모 코드 실행

    def walk(self):
        # 새로운 메소드 추가
        print(f"{self.name}가 산책을 합니다")

# 사용 예시
dog = Dog("멍멍이")
dog.make_sound()
dog.walk()
# 출력: 멍멍이가 멍멍!
# 출력: 멍멍이가 산책을 합니다

```

- 자식 클래스에서 새로운 메소드를 추가할 수 있습니다.
- 자식 클래스에서 부모의 메소드를 재정의할 수 있습니다. (**Overriding**)

▼ 미니 실습

▼ 밴드 만들기

1. **Musician** 클래스를 만들고 **play()** 메소드를 선언하세요
2. Guitarist, Drummer, Pianist 등의 클래스를 만들고, **Musician** 을 상속 받습니다.
3. **band 리스트**를 만들고, 각각의 연주자들을 생성하여 넣습니다.
4. band를 for 문으로 돌면서 각 연주자들을 **play()** 시킵니다.
5. 아래와 같이 출력되어야 합니다:
 - "기타 징징징"
 - "드럼 우당탕탕"
 - "피아노 팡팡팡"
 - "베이스 둥둥둥"

▼ 코드 예시

```

# 부모 클래스 정의
class Musician:
    def __init__(self, name):
        self.name = name

    def play(self):
        print("음악을 연주한다!")

class Guitarist(Musician):
    # play를 기타리스트만의 방식으로 재정의
    def play(self):
        print("기타 징징징!")

class Pianist(Musician):
    # play를 피아니스트 방식으로 재정의
    def play(self):
        print("피아노 땡땡땡!")

class Drummer(Musician):
    # play를 드러머만의 방식으로 재정의
    def play(self):
        print("드러머는 우당탕탕!")

guitarist = Guitarist("김기타")
pianist = Pianist("이피아니스트")
drummer = Drummer("박드러머")

# 밴드 구성하기
band = [guitarist, pianist, drummer]

# 밴드 연주하기
for musician in band:
    musician.play()

```




부모 클래스를 참조하려면 어떻게 해야하나요?

자식 클래스에서 부모 클래스의 메소드나 속성을 참조하려면 `super()` 함수를 사용합니다.

```
class Dog(Animal):
    def make_sound(self):
        super().make_sound() # 부모 클래스의 make_sound() 호출
```



여러 개의 부모 클래스를 상속 받을 수도 있나요?

네, 다중상속(Multiple Inheritance)도 가능합니다.

하지만 다중 상속은 복잡성을 증가시킬 수 있어 신중하게 사용해야 합니다.

또한 다른 언어에서는 다중 상속이 안되는 경우가 많으니 참고하세요.

```
class Singer:
    def sing(self):
        print("노래를 부릅니다")

class Dancer:
    def dance(self):
        print("춤을 춥니다")

# 가수와 댄서의 특성을 모두 상속받는 아이돌
class Idol(Singer, Dancer):
    pass

idol = Idol()
idol.sing() # 출력: 노래를 부릅니다
idol.dance() # 출력: 춤을 춥니다
```

▼ 다형성(Polymorphism)

상속받은 여러 클래스들이 **같은 메소드명을 사용하지만, 각자 다르게 동작하는** 것을 뜻합니다.

▼ 다형성의 예시

```
# 부모 클래스
class Musician:
    def play(self):
        print("연주를 합니다")

# 자식 클래스들
class Guitarist(Musician):
    def play(self):
        print("기타 연주: 징징징")

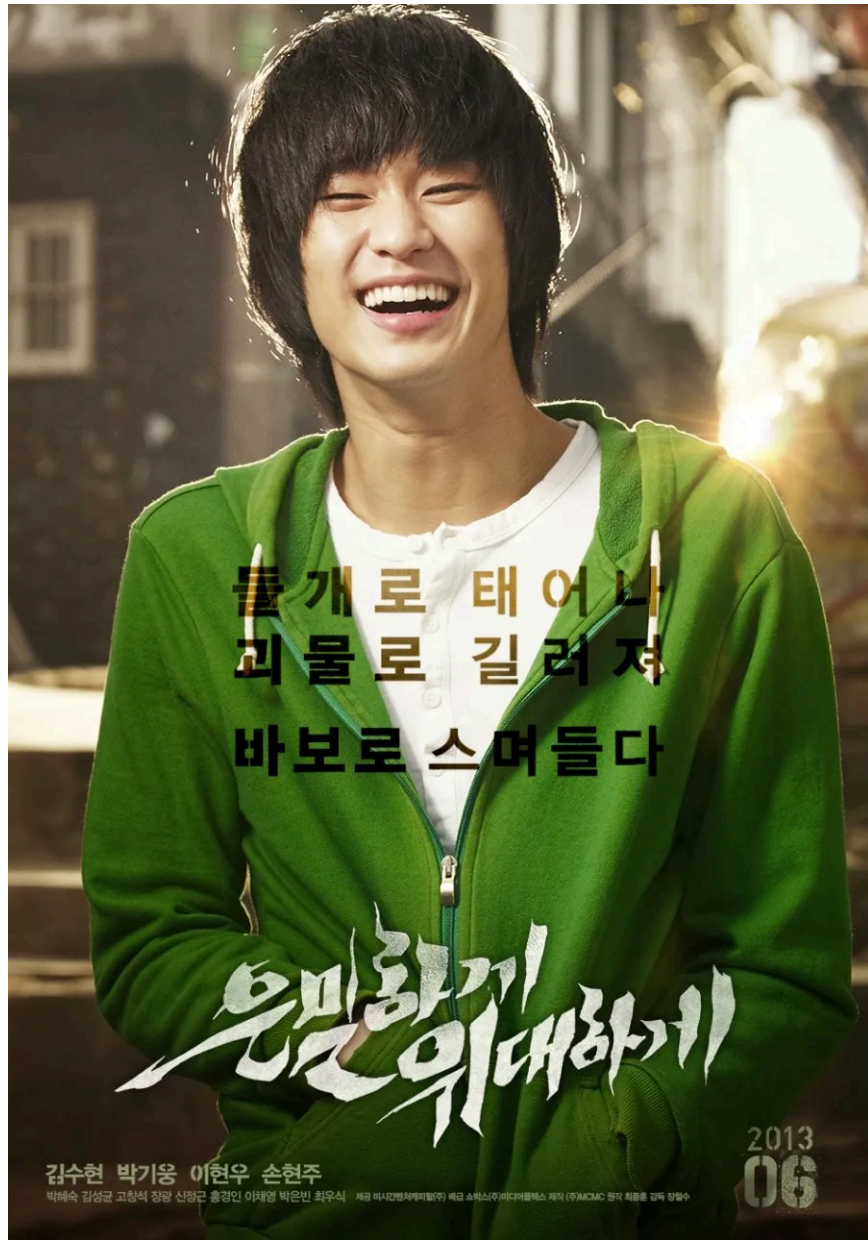
class Drummer(Musician):
    def play(self):
        print("드럼 연주: 쿵짝쿵짝")

class Pianist(Musician):
    def play(self):
        print("피아노 연주: 땡땡땡")

# 다형성 활용
band = [Guitarist(), Drummer(), Pianist()]
for musician in band:
    musician.play()
```

- 음악가들한테 '**연주하라**'는 요청을 하면, **각기 다른 악기를 연주**를 하는 것과 같습니다.
- 피아니스트에게 '피아노를 연주하세요'라고 구체적으로 말하지 않아도 '**연주하세요**'라는 말만으로 충분합니다.
- 이처럼 상속 구조를 잘 설계하면, **같은 이름의 메소드**로 다양한 음악가들에게 **각각 다른 동작을 수행**하도록 할 수 있습니다.

▼ 캡슐화(Capsulization)



캡슐화는 객체의 내부 데이터와 구현을 숨기고,
외부에서는 허용된 방법으로만 접근할 수 있도록 하는 것입니다.

```
# 캡슐화가 적용되지 않은 클래스
class BankAccount:
    def __init__(self, balance):
        self.balance = balance # 누구나 직접 접근 가능

# 캡슐화가 적용된 클래스
class SafeBankAccount:
```

```

def __init__(self, balance):
    self.__balance = balance

# 잔액 조회
def get_balance(self, username):
    if username == "김기범":
        return self.__balance
    else:
        return -1

# 출금
def withdraw(self, username, amount):
    if username != "김기범":
        print("출금 권한이 없는 유저입니다.")
    elif self.__balance <= amount:
        print("잔액이 충분하지 않습니다")
    else:
        self.__balance -= amount
        print(f"현재 잔액은 {self.__balance}원 입니다.")
        return amount

```

- 파이썬에서는 변수명 앞에 **__**를 붙여서 private 변수를 만듭니다.
- 외부에서는 **메소드를 통해서만** 데이터에 접근할 수 있습니다.
- 은행 금고처럼, 중요한 데이터는 직접 접근할 수 없고,
정해진 절차(메소드)를 통해서만 접근할 수 있게 함으로써 데이터를 안전하게 보호할 수 있습니다.

▼ 실습

은행 계좌 관리 시스템 만들기

1. BankAccount 클래스를 만들고 다음 private 변수들을 선언하세요:

- __account_number (계좌번호)
- __balance (잔액)
- __owner (계좌주)

2. 다음 public 메소드들을 구현하세요:

- deposit(amount): 입금
- withdraw(amount): 출금
- get_balance(): 잔액 조회
- get_account_info(): 계좌 정보 조회

3. 입금과 출금시 다음 규칙을 적용하세요:

- 입금액과 출금액은 0보다 커야 함
- 출금시 잔액이 부족하면 거래 거절

▼ 코드 예시

```
class BankAccount:
    def __init__(self, account_number, owner):
        self.__account_number = account_number
        self.__balance = 0
        self.__owner = owner

    def deposit(self, amount):
        if amount <= 0:
            print("입금액은 0보다 커야 합니다.")
            return False
        self.__balance += amount
        print(f"{amount}원이 입금되었습니다.")
        return True

    def withdraw(self, amount):
        if amount <= 0:
            print("출금액은 0보다 커야 합니다.")
            return False
        if self.__balance < amount:
            print("잔액이 부족합니다.")
            return False
        self.__balance -= amount
        print(f"{amount}원이 출금되었습니다.")
        return True

    def get_balance(self):
        return self.__balance
```

```
def get_account_info(self):
    return {
        "계좌번호": self.__account_number,
        "계좌주": self.__owner,
        "잔액": self.__balance
    }

# 사용 예시
account = BankAccount("1234-5678", "김철수")
account.deposit(50000)
account.withdraw(30000)
print(account.get_account_info())
```